Praktikum 6
Jaringan Kohonen

NAMA	: Ihsanta Zaki Sanjaya
KELAS	: 2 D4 Teknik Komputer B
NRP	: 3223600058
DOSEN	: Riyanto Sigit, ST, M.Kom, Ph.D
MATA KULIAH	: Praktikum Sistem Cerdas
TANGGAL	: Rabu, 26 Maret 2025

D4 TEKNIK KOMPUTER
POLITEKNIK ELEKTRONIKA NEGERI SURABAYA
2025

Praktikum 6

Membuat Program Aplikasi Jaringan Kohonen

A. Tujuan

1. Mahasiswa dapat memahami dan menjelaskan konsep Jaringan Kohonen
2. Mahasiswa dapat menjelaskan model Jaringan Kohonen
3. Mahasiswa dapat membuat aplikasi Jaringan

Kohonen Software yang diperlukan:

- Microsoft Visual C++
- PyCharm

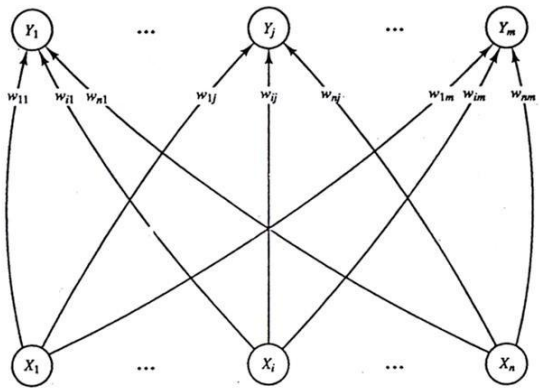
B. Pendahuluan

1. Jaringan Kohonen

Jaringan kohonen ini pertama kali diperkenalkan oleh Prof. Teuvo Kohonen pada tahun 1982. Pada jaringan ini, suatu lapisan yang berisi neuron-neuron akan menyusun dirinya sendiri berdasarkan input nilai tertentu dalam suatu kelompok yang dikenal dengan istilah cluster. Selama proses penyusunan diri, cluster yang memiliki vektor bobot paling cocok dengan pola input (memiliki jarak yang paling dekat) akan terpilih sebagai pemenang. Neuron yang menjadi pemenang beserta neuron-neuron tetangganya akan memperbaiki bobot-bobotnya.

Kohonen adalah merupakan algoritma jaringan cerdas dengan kategori pembelajaran secara kompetitif dan bersifat *unsupervised*. Sistem secara otomatis dapat melakukan pengelompokan atau klasifikasi tanpa menggunakan pembelajaran dengan pasangan data terlebih dahulu. Secara umum, pembaruan nilai bobot adalah berdasar nilai jarak terkecil dari bobot terhadap nilai data masukan. Pembaharuan dilakukan hanya pada bobot yang berhubungan dengan node yang terdekat

2. Arsitektur Jaringan Kohonen



Gambar 1. Arsitektur Jaringan Kohonen

- Xi : input data
- Wij : bobot
- Yi : output cluster

3. Algoritma Jaringan Kohonen

- Inisialisasi Bobot Wij
- Inisialisasi jumlah cluster yang akan dibentuk
- Atur nilai Learning Rate
- Inisialisasi input vektor yang akan diklasifikasikan

1. **Hitung nilai D(j) yang merupakan jarak antara bobot dengan data masukan vektor. D(j) =** $\sum \boldsymbol{i}\ (\boldsymbol{Wij}\ - \boldsymbol{Xi})^2$
2. **Temukan D(j) minumum dari perhitungan pada kedua bobot tersebut**
3. **Lakukan pembaruan pada bobot yang memiliki nilai D(j) minimum Wij(new) = Wij (old) + α (Xi – Wij (old))**
4. **Terdapat bobot baru setelah pembaruan tersebut.**
5. **Ulangi langkah 1 hingga 4 untuk vektor selanjutnya**
6. **Update Learning Rate**
7. **Proses pembelajaran ini akan berlangsung terus hingga epochnya mencapai maksimum epoch.**
8. **Proses iterasi berhenti jika bobot baru dengan bobot lama selisihnya adalah nol.**

4. Contoh Aplikasi Jaringan Kohonen

1. Analisis metode kohonen dalam mengenali pola tanda tangan
2. Metode kohonen dalam clustering kualitas air di kali Surabaya
3. Analisis metode kohonen dalam mengenali karakter huruf arab
4. Metode kohonen dalam analisis cluster gaya belajar mahasiswa di jawa barat
5. Metode kohonen untuk visualisasi data geospasial pada informasi sebaran DPT atau data pemilih tetap
6. Metode kohonen dalam memetakan daerah rawan bencana alam tanah longsor di Indonesia
7. Metode kohonen dalam analisis pariwisata di Indonesia
8. Metode kohonen analisis korelasi antara penalaran matematika dengan nilai kelulusan mahasiswa di sumatera barat.

5. Contoh Implementasi Jaringan Kohonen

- Misalkan kita ingin mengcluster data-data berikut:

	X1	X2	X3	X4
1.	1	1	0	0
2.	0	0	0	1
3.	1	0	0	0
4.	0	0	1	1

- Menjadi 2 cluster. Learning rate (α=0,6) dengan tiap kenaikan epoh akan diset 0,5x(α).
- Bobot awal yang akan kita gunakan adalah matriks berukuran 2x4.
  - W[2][4]={0.2, 0.6, 0.5, 0.9},
  - {0.8, 0.4, 0.7, 0.3}};

6. Testing

Misalkan kita akan melakukan testing terhadap data ketiga: [1, 1, 0, 0] termasuk dalam cluster mana?

7. Contoh Program

```
#include "stdafx.h"
#include <conio.h>
#include  <math.h>
void main(){
            float w[2][4]={{0.2,0.6,0.5,0.9},
                           {0.8,0.4,0.7,0.3}};
            int x[4][4]={{1,1,0,0},
                         {0,0,0,1},
                         {1,0,0,0},
                         {0,0,1,1}};
           float D[2];
           int i,j,k,l;
           float LR=0.6;
           //training
           for(l=0;l<100;l++){
           for(k=0;k<4;k++){
                for(j=0;j<2;j++){
                     D[j]=0.0;
             for(i=0;i<4;i++){
                      D[j]=D[j]+(w[j][i]-x[k][i])*(w[j][i]-x[k][i]);
             }
                }
                if(D[0]<D[1]){
                     for(i=0;i<4;i++){
                      w[0][i]=w[0][i]+LR*(x[k][i]-w[0][i]);
                     }
                }else{
                     for(i=0;i<4;i++){
                      w[1][i]=w[1][i]+LR*(x[k][i]-w[1][i]);
                     }
                }
               }
               LR=0.5*LR;
           }
           //mapping cluster
           x[0][0]=1; x[0][1]=1; x[0][2]=0; x[0][3]=0;
           for(j=0;j<2;j++){
               D[j]=0.0;
               for(i=0;i<4;i++){
                      D[j]=D[j]+sqrt((w[j][i]-x[0][i])*(w[j][i]-x[0][i]));
               }
               printf("D(%d) = %.2f\n",(j+1),D[j]);
           }
           if(D[0]<D[1]){
               printf("Cluster 1");
           }else{
               printf("Cluster 2");
           }
}
```

Program:

```
#include <conio.h>
#include <math.h>
#include <stdio.h>

int main() {
    float w[2][4] = {
        {0.2, 0.6, 0.5, 0.9},
        {0.8, 0.4, 0.7, 0.3}
    };

    int x[4][4] = {
        {1, 1, 0, 0},
        {0, 0, 0, 1},
        {1, 0, 0, 0},
        {0, 0, 1, 1}
    };

    float D[2];
    int i, j, k, l;
    float LR = 0.6;

    // Training phase
    for (l = 0; l < 100; l++) {
        for (k = 0; k < 4; k++) {
            for (j = 0; j < 2; j++) {
                D[j] = 0.0;
                for (i = 0; i < 4; i++) {
                    D[j] += (w[j][i] - x[k][i]) * (w[j][i] - x[k][i]);
                }
            }

            if (D[0] < D[1]) {
                for (i = 0; i < 4; i++) {
                    w[0][i] = w[0][i] + LR * (x[k][i] - w[0][i]);
                }
            } else {
                for (i = 0; i < 4; i++) {
                    w[1][i] = w[1][i] + LR * (x[k][i] - w[1][i]);
                }
            }
        }
        LR = 0.5 * LR;
    }

    // Mapping cluster
    x[0][0] = 1;
    x[0][1] = 1;
    x[0][2] = 0;
```

```
    x[0][3] = 0;

    for (j = 0; j < 2; j++) {
        D[j] = 0.0;
        for (i = 0; i < 4; i++) {
            D[j] += sqrt((w[j][i] - x[0][i]) * (w[j][i] - x[0][i]));
        }
        printf("D(%d) = %.2f\n", (j + 1), D[j]);
    }

    if (D[0] < D[1]) {
        printf("Cluster 1\n");
    } else {
        printf("Cluster 2\n");
    }
return 0;
}
```

8. Output

Microsoft Visual Studio Debug Console

```
D(1) = 3.54
D(2) = 0.64
Cluster 2
D:\PENS\Tekkom\Sistem Cerdas\2021 Siste
xited with code 0.
To automatically close the console when
le when debugging stops.
Press any key to close this window . .
```

Hasil:

```
PS D:\1. KULIAH\Semester IV
D(1) = 3.54
D(2) = 0.64
Cluster 2
```

9. Tugas

Misalkan kita ingin mengcluster data-data berikut:

X_1	X_2
0,10	0,10
0,20	0,20
0,30	0,10
0,50	0,30
0,40	0,40
0,20	0,40

menjadi 2 cluster. Bobot awal yang akan kita gunakan adalah matriks berukuran 2x2 dengan tiap-tiap elemen bernilai 0,5. Learning rate (α=0,6) dengan tiap kenaikan epoh akan diset 0,5x(α). Maksimum epoh ditetapkan sebesar 10.

Program:

```
#include <stdio.h>
#include <math.h>

int main() {

float w[2][2] = {{0.5, 0.5},
{0.5, 0.5}};

float x[6][2] = {{0.10, 0.10},
{0.20, 0.20},
{0.30, 0.10},
{0.50, 0.30},
{0.40, 0.40},
{0.20, 0.40}};

float D[2];
int i, j, k, l;
float LR = 0.6;
int epoch_max = 10;

// Training
for (l = 0; l < epoch_max; l++) {
for (k = 0; k < 6; k++) {
for (j = 0; j < 2; j++) {
D[j] = 0.0;
for (i = 0; i < 2; i++) {
D[j] += (w[j][i] - x[k][i]) * (w[j][i] - x[k][i]);
}
}
printf("Data %.2f, %.2f: D[0] = %.4f, D[1] = %.4f\n", x[k][0],
x[k][1], D[0],

D[1]);

if (D[0] < D[1]) {
for (i = 0; i < 2; i++) {
w[0][i] += LR * (x[k][i] - w[0][i]);
}
} else {
for (i = 0; i < 2; i++) {

w[1][i] += LR * (x[k][i] - w[1][i]);
}
}
}
LR *= 0.5;
}
printf("\nHasil Pengelompokan:\n");
for (k = 0; k < 6; k++) {
for (j = 0; j < 2; j++) {
```

```
D[j] = 0.0;

for (i = 0; i < 2; i++) {
D[j] += (w[j][i] - x[k][i]) * (w[j][i] - x[k][i]);
}
}
printf("Data %.2f, %.2f: D[0] = %.4f, D[1] = %.4f\n", x[k][0],
x[k][1], D[0], D[1]);
if (D[0] < D[1]) {
printf("Data %.2f, %.2f berada di Cluster 1\n", x[k][0], x[k][1]);
} else {
printf("Data %.2f, %.2f berada di Cluster 2\n", x[k][0], x[k][1]);
}
}

return 0; }
```

Hasil training:

```
Data 0.30, 0.10: D[0] = 0.2000, D[1] = 0.0212
Data 0.50, 0.30: D[0] = 0.0400, D[1] = 0.0757
Data 0.40, 0.40: D[0] = 0.0104, D[1] = 0.0797
Data 0.20, 0.40: D[0] = 0.0577, D[1] = 0.0675
Data 0.10, 0.10: D[0] = 0.1265, D[1] = 0.0312
Data 0.20, 0.20: D[0] = 0.0479, D[1] = 0.0046
Data 0.30, 0.10: D[0] = 0.0881, D[1] = 0.0105
Data 0.50, 0.30: D[0] = 0.0510, D[1] = 0.0943
Data 0.40, 0.40: D[0] = 0.0029, D[1] = 0.0945
Data 0.20, 0.40: D[0] = 0.0294, D[1] = 0.0702
Data 0.10, 0.10: D[0] = 0.1287, D[1] = 0.0208
Data 0.20, 0.20: D[0] = 0.0481, D[1] = 0.0049
Data 0.30, 0.10: D[0] = 0.0811, D[1] = 0.0089
Data 0.50, 0.30: D[0] = 0.0398, D[1] = 0.1007
Data 0.40, 0.40: D[0] = 0.0037, D[1] = 0.0992
Data 0.20, 0.40: D[0] = 0.0244, D[1] = 0.0704
Data 0.10, 0.10: D[0] = 0.1315, D[1] = 0.0177
Data 0.20, 0.20: D[0] = 0.0494, D[1] = 0.0048
Data 0.30, 0.10: D[0] = 0.0791, D[1] = 0.0083
Data 0.50, 0.30: D[0] = 0.0348, D[1] = 0.1036
Data 0.40, 0.40: D[0] = 0.0039, D[1] = 0.1011
Data 0.20, 0.40: D[0] = 0.0225, D[1] = 0.0705
Data 0.10, 0.10: D[0] = 0.1330, D[1] = 0.0165
Data 0.20, 0.20: D[0] = 0.0502, D[1] = 0.0047
Data 0.30, 0.10: D[0] = 0.0783, D[1] = 0.0081
Data 0.50, 0.30: D[0] = 0.0326, D[1] = 0.1049
Data 0.40, 0.40: D[0] = 0.0039, D[1] = 0.1021
Data 0.20, 0.40: D[0] = 0.0217, D[1] = 0.0705
Data 0.10, 0.10: D[0] = 0.1338, D[1] = 0.0159
Data 0.20, 0.20: D[0] = 0.0506, D[1] = 0.0046
Data 0.30, 0.10: D[0] = 0.0779, D[1] = 0.0079
Data 0.50, 0.30: D[0] = 0.0315, D[1] = 0.1055
Data 0.40, 0.40: D[0] = 0.0039, D[1] = 0.1025
Data 0.20, 0.40: D[0] = 0.0213, D[1] = 0.0705
Data 0.10, 0.10: D[0] = 0.1342, D[1] = 0.0157
Data 0.20, 0.20: D[0] = 0.0508, D[1] = 0.0046
Data 0.30, 0.10: D[0] = 0.0778, D[1] = 0.0079
Data 0.50, 0.30: D[0] = 0.0310, D[1] = 0.1059
Data 0.40, 0.40: D[0] = 0.0039, D[1] = 0.1027
Data 0.20, 0.40: D[0] = 0.0211, D[1] = 0.0705
Data 0.10, 0.10: D[0] = 0.1344, D[1] = 0.0155
Data 0.20, 0.20: D[0] = 0.0509, D[1] = 0.0046
Data 0.30, 0.10: D[0] = 0.0777, D[1] = 0.0078
Data 0.50, 0.30: D[0] = 0.0307, D[1] = 0.1060
Data 0.40, 0.40: D[0] = 0.0039, D[1] = 0.1029
Data 0.20, 0.40: D[0] = 0.0210, D[1] = 0.0706
Data 0.10, 0.10: D[0] = 0.1345, D[1] = 0.0155
Data 0.20, 0.20: D[0] = 0.0510, D[1] = 0.0046
Data 0.30, 0.10: D[0] = 0.0776, D[1] = 0.0078
Data 0.50, 0.30: D[0] = 0.0306, D[1] = 0.1061
Data 0.40, 0.40: D[0] = 0.0039, D[1] = 0.1029
Data 0.20, 0.40: D[0] = 0.0209, D[1] = 0.0706
Data 0.10, 0.10: D[0] = 0.1345, D[1] = 0.0154
Data 0.20, 0.20: D[0] = 0.0510, D[1] = 0.0046
Data 0.30, 0.10: D[0] = 0.0776, D[1] = 0.0078
Data 0.50, 0.30: D[0] = 0.0306, D[1] = 0.1061
Data 0.40, 0.40: D[0] = 0.0039, D[1] = 0.1029
Data 0.20, 0.40: D[0] = 0.0209, D[1] = 0.0706
```

Hasil testing:

```
Hasil Pengelompokan:
Data 0.10, 0.10: D[0] = 0.1346, D[1] = 0.0154
Data 0.10, 0.10 berada di Cluster 2
Data 0.20, 0.20: D[0] = 0.0510, D[1] = 0.0046
Data 0.20, 0.20 berada di Cluster 2
Data 0.30, 0.10: D[0] = 0.0776, D[1] = 0.0078
Data 0.30, 0.10 berada di Cluster 2
Data 0.50, 0.30: D[0] = 0.0305, D[1] = 0.1061
Data 0.50, 0.30 berada di Cluster 1
Data 0.40, 0.40: D[0] = 0.0039, D[1] = 0.1029
Data 0.40, 0.40 berada di Cluster 1
Data 0.20, 0.40: D[0] = 0.0209, D[1] = 0.0706
Data 0.20, 0.40 berada di Cluster 1
```

C. Analisa

Pada praktikum ini, jaringan Kohonen digunakan untuk melakukan clustering data dengan pendekatan pembelajaran tanpa supervisi. Dua program yang diuji menggunakan dataset yang berbeda, dengan tujuan untuk mengevaluasi kemampuan jaringan Kohonen dalam mengelompokkan data berdasarkan pola yang ditemukan secara otomatis. Dalam program percobaan pertama, dataset biner dengan empat fitur digunakan, dan jaringan Kohonen mengklasifikasikan data ke dalam dua cluster berdasarkan jarak Euclidean antara data dan bobot cluster. Proses pelatihan dilakukan selama 100 iterasi, dengan learning rate (LR) yang berkurang setiap epoch untuk meningkatkan stabilitas pembelajaran. Program ini juga mengklasifikasikan input baru setelah proses pelatihan berdasarkan jarak terdekat ke bobot cluster.

Di sisi lain, program tugas menggunakan dataset dua dimensi dengan enam titik data yang terus diperbarui bobotnya selama 10 iterasi. Setiap bobot diperbarui mengikuti prinsip yang sama, namun dengan perbedaan pada jumlah iterasi dan dataset yang digunakan. Program tugas juga memberikan lebih banyak umpan balik selama pelatihan, seperti menampilkan bobot setiap epoch yang membantu untuk memantau konvergensi. Meskipun keduanya menggunakan algoritma yang serupa, perbedaan dalam jumlah iterasi, dataset, dan fitur interaktif dalam program tugas menunjukkan variasi dalam hasil yang diperoleh.

Jaringan Kohonen sendiri menggunakan konsep pembelajaran kompetitif, di mana bobot neuron diperbarui berdasarkan kedekatannya dengan input data. Pembaruan bobot ini mengarah pada pembentukan cluster yang lebih baik seiring waktu. Namun, hasil clustering sangat dipengaruhi oleh parameter seperti learning rate dan posisi bobot awal. Oleh karena itu, pemilihan parameter yang tepat sangat krusial untuk mencapai hasil yang optimal dalam pengelompokan data. Secara keseluruhan, meskipun jaringan Kohonen memiliki kelemahan dalam hal sensitivitas terhadap parameter awal, jaringan ini terbukti efektif dalam mengelompokkan data berdasarkan kemiripan pola.

D. Kesimpulan

Jaringan Kohonen berhasil mengelompokkan data menggunakan pendekatan pembelajaran tanpa supervisi, di mana hasil clustering sangat dipengaruhi oleh parameter seperti learning rate, jumlah iterasi, dan bobot awal. Program yang diuji menunjukkan bahwa jaringan dapat secara otomatis membentuk cluster berdasarkan kedekatan jarak input dengan bobot cluster, meskipun sensitivitas terhadap parameter awal dapat mempengaruhi hasil. Program tugas dengan fitur interaktif memberikan lebih banyak umpan balik dan memungkinkan eksplorasi lebih dalam terhadap konvergensi bobot. Secara keseluruhan, jaringan Kohonen efektif untuk clustering data sederhana, namun dapat dioptimalkan lebih lanjut untuk aplikasi yang lebih kompleks dengan penyesuaian parameter.